

PCOD

FORMAT DESCRIPTION AND IMPLEMENTATION GUIDE

PCOD Hybrid Database 0.2 - English Edition

Format name	PCOD Hybrid Database
Format version	0.2
Reference software	PCOD 0.6.23
Author / Copyright holder	SpiderPawn Việt Nam
Document license	MIT License
Document edition	1.0 - 2026

A technical specification and implementation guide for developers, AI tools, and database administrators.

Contents

This contents list is intentionally static so the PDF remains reliable across viewers.

- 1 Scope, audience, and license**
- 2 The format in 60 seconds**
- 3 The .pcod directory bundle**
- 4 manifest.properties**
- 5 games.pci - fixed hot index**
- 6 names.pcn - name dictionary**
- 7 games.pcg - payload stream**
- 8 Payload codecs**
- 9 Reconstructing complete PGN**
- 10 Audio assets and H2 sidecar**
- 11 Safe read workflow**
- 12 Import transactions and crash recovery**
- 13 Update, soft delete, and compaction**
- 14 Validation and data safety**
- 15 Format conversion**
- 16 Python index-reader skeleton**
- 17 Cheat sheets and troubleshooting**
- 18 Future compatibility**
- 19 MIT License**

1. Scope, audience, and license

This guide describes PCOD Hybrid Database format version 0.2 as implemented and cross-checked against PCOD 0.6.23. Its purpose is to let a new developer, validator, conversion tool, or AI assistant understand the storage model without guessing from file extensions or reading the complete PCOD source tree.

1.1. Intended audience

- **Read-only reader developers:** search, inspect, or export games without modifying the database.
- **Writer and importer developers:** implement durable transactions, rollback, and crash tests.
- **Validation tools:** detect truncation, CRC failures, missing name IDs, malformed payloads, and unsafe paths.
- **AI tools:** use an authoritative layout description instead of inventing binary structures.
- **Database administrators:** back up, move, recover, and compact PCOD bundles safely.

1.2. Author and permission to use

Item	Information
Author / copyright holder	SpiderPawn Việt Nam
Copyright year	2026
Documentation license	MIT License
Licensed material	This guide and associated format-specification files
Not automatically covered	PCOD source code, SDKs, logos, trademarks, or other software components unless separately licensed

Legal note: The MIT grant in this guide applies to the documentation and associated specification files. Software, source code, and brand assets remain subject to their own licenses.

1.3. Non-negotiable rules

- A `.pcod`` database is a directory bundle, not a single file.
- ``games.pci`` has a 64-byte header and fixed 56-byte records in format 0.2.
- Game ID is record ordinal + 1, starts at 1, and is never reused.
- Deletion is a soft-delete flag; the physical record remains.
- ``games.pcg`` is append-only; updates append a new payload and redirect the index.
- Verify payload CRC32 before decoding.
- Handle mixed endianness exactly as specified.
- Do not write without recovery markers, force/flush ordering, and crash tests.
- Reject or route to another reader when ``formatVersion`` is not ``0.2``.

2. The format in 60 seconds

PCOD Hybrid 0.2 separates hot searchable metadata, repeated names, and complete game content.

```
Fast searchable metadata -> games.pci
Repeated names          -> names.pcn
Complete game content   -> games.pcg
```

2.1. Reading one game

```

Game ID
|
v
games.pci: read one 56-byte record
|
+--> names.pcn: resolve whiteId/blackId/eventId/siteId
|
+--> games.pcg: read payloadOffset + payloadLength
|
+--> verify CRC32
+--> decode MOVE16 or PGN_DEFLATE
+--> reconstruct complete PGN

```

2.2. Import flow

```

PGN -> ParsedGame -> name IDs + packed metadata
-> append payload to games.pcg
-> append uncommitted index record to games.pci
-> atomically save names.pcn
-> commit gameCount in the games.pci header

```

Commit point: The `gameCount` value at offset 16 of the `games.pci` header is the final import commit point. Physical records beyond the committed count are uncommitted.

3. The .pcod directory bundle

```

MyDatabase.pcod/
|-- manifest.properties      required
|-- games.pci               required
|-- names.pcn               present after names exist
|-- games.pcg               required
|-- extras.mv.db             optional H2 sidecar
|-- sound/                  optional
|   |-- <sha256>.mp3
|   |-- <sha256>.flac
|   |-- <sha256>.ogg
|-- recovery/                transaction markers
|   |-- import.properties
|   |-- update.properties

```

Component	Role	Required	Back up
manifest.properties	Format, version, and codec identity	Yes	Yes
games.pci	Fixed index, Game IDs, hot metadata	Yes	Yes
names.pcn	Player/Event/Site dictionary	When names exist	Yes
games.pcg	Complete payloads	Yes	Yes
extras.mv.db	Import history and user notes	No	Recommended
sound/	Audio assets referenced by movetext	No	Yes when used
recovery/	In-progress transaction markers	Temporary	Preserve during incidents

3.1. Temporary files during atomic writes

```

manifest.properties.tmp
names.pcn.tmp
recovery/import.properties.tmp
recovery/update.properties.tmp

```

A `.tmp`` file is not committed data. For consistent backups, stop writers or use a filesystem snapshot that captures the directory atomically.

3.2. What is not part of the database

Extensions, themes, piece sets, fonts, and language packs live in application configuration. They are not part of PCOD Hybrid 0.2 and must not be included in core database-reader logic.

4. manifest.properties

This Java Properties file identifies the database before any binary file is interpreted.

```
format=PCOD-HYBRID
formatVersion=0.2
databaseId=550e8400-e29b-41d4-a716-446655440000
createdAt=2026-06-14T10\:20\:30.123Z
indexRecordSize=56
payloadCodecs=MOVE16_V1,PGN_DEFLATE_V1
```

Key	Value / meaning
format	Must be <code>`PCOD-HYBRID`</code>
formatVersion	Must be <code>`0.2`</code> for this guide
databaseId	Database UUID
createdAt	ISO-8601 creation time
indexRecordSize	56
payloadCodecs	MOVE16_V1,PGN_DEFLATE_V1

4.1. Compatibility rules

- Reject a different ``format``.
- Reject a different ``formatVersion``.
- Reject an explicit ``indexRecordSize`` other than 56.
- Reject malformed or out-of-range numeric values.
- Ignore unknown keys for forward extensibility unless they alter mandatory boundaries or codecs.

5. games.pci - fixed hot index

``games.pci`` supports direct record lookup by Game ID. The entire file uses little-endian byte order.

```
recordOffset(gameId) = 64 + (gameId - 1) * 56
expectedFileLength = 64 + committedGameCount * 56
```

5.1. The 64-byte header

Offset	Size	Type	Content
0	4	int32 LE	numeric magic 0x50434931 (PCI1)
4	2	uint16 LE	version = 1
6	2	uint16 LE	record size = 56
8	4	int32 LE	header size = 64
12	4	int32 LE	reserved, currently 0
16	8	int64 LE	committed game count

Offset	Size	Type	Content
24	8	int64 LE	reserved, commonly initialized to 1
32	8	int64 LE	creation time in Unix epoch milliseconds
40	24	bytes	reserved

Magic and byte order: The Java constant is ``0x50434931``, but the file is little-endian. Decode an LE int32 and compare the numeric value; do not search raw bytes for an ASCII string.

5.2. The 56-byte record

Offset	Size	Type	Field
0	8	int64 LE	payloadOffset
8	4	int32 LE	payloadLength
12	4	int32 LE	whiteId
16	4	int32 LE	blackId
20	4	int32 LE	eventId
24	4	int32 LE	siteId
28	4	uint32 LE	packedDate
32	4	uint32 LE	packedElo
36	4	uint32 LE	packedMeta
40	4	int32 LE	plyCount
44	8	uint64 LE	semanticHash
52	4	uint32 LE	payloadCrc

5.3. packedDate

Bits	Mask	Meaning
0-13	0x00003FFF	Year; 0 means unknown
14-17	0x0003C000	Month; 0 means unknown
18-22	0x007C0000	Day; 0 means unknown
23-24	0x01800000	Precision: 0 none, 1 year, 2 month, 3 day
25-31	-	Reserved

```

year      = packedDate & 0x3FFF
month     = (packedDate >>> 14) & 0x0F
day       = (packedDate >>> 18) & 0x1F
precision = (packedDate >>> 23) & 0x03

```

A complete reader should prefer the exact Date string stored in the payload. ``packedDate`` primarily supports fast search and fallback output.

5.4. packedElo and packedMeta

```
whiteEloRaw = packedElo & 0xFFFF
blackEloRaw = (packedElo >>> 16) & 0xFFFF
```

packedMeta bits	Meaning
0-9	Packed ECO
10-11	Result code
12	FLAG_DELETED
13	FLAG_COMMENTS
14	FLAG_VARIATIONS
15	FLAG_START_FEN
16	FLAG_MOVE16
17-31	Reserved

Result code	Result
0	* or unknown
1	1-0
2	0-1
3	1/2-1/2

Packed ECO

0 means unknown. Values 1..500 encode A00..E99:

```
packed = 1 + (letter - 'A') * 100 + number
n      = packed - 1
letter = 'A' + n / 100
number = n % 100
```

5.5. plyCount, semanticHash, and payloadCrc

- **plyCount**: exact for MOVE16; an estimated main-line count for PGN_DEFLATE.
- **semanticHash**: FNV-1a 64-bit content identity hint, not an integrity checksum or security signature.
- **payloadCrc**: standard CRC32 over the complete payload bytes; verify before decoding.

```
trim(White) + U+001F + trim(Black) + U+001F + trim(Date)
+ U+001F + trim(Result) + U+001F
+ movetext with consecutive whitespace collapsed and trimmed
```

5.6. Soft deletion and file length

```
packedMeta |= 1 << 12
```

- If the file is shorter than `64 + gameCount \* 56`, it is truncated and must fail validation.
- If it is longer than the committed length, the tail is uncommitted records or crash residue.
- Do not derive committed gameCount from physical file size.
- Never reuse a deleted Game ID.

6. names.pcn - name dictionary

`names.pcn` deduplicates Player, Event, and Site strings. Fixed-width fields are big-endian; varints use low 7-bit groups first.

6.1. Header and section order

Offset	Size	Type	Content
0	4	int32 BE	magic 0x50434E31, raw ASCII PCN1
4	2	uint16 BE	version = 1
6	2	uint16 BE	reserved = 0

PLAYER
EVENT
SITE

6.2. Section structure

```
int32 BE count
repeat count times:
    unsigned-varint stableId
    unsigned-varint commonPrefixLength
    unsigned-varint suffixByteLength
    byte[suffixByteLength] UTF-8 suffix
```

6.3. Front coding and UTF-16

```
value = previous.substring(0, commonPrefixLength) + UTF8(suffix)
```

Most common interoperability bug: `commonPrefixLength` is measured in Java UTF-16 code units (`String.length()`), not UTF-8 bytes, Unicode code points, or grapheme clusters. Supplementary characters may occupy two units.

6.4. Stable IDs are not physical positions

- Every entry explicitly stores its `stableId`.
- Physical entries are sorted by name for front coding, not by ID.
- Never assume entry N has ID N.
- ID 0 is not a real name; invalid lookups commonly become `?`.
- An index record is meaningful only with its matching `names.pcn`.

6.5. Duplicate normalization

1. Map null/empty to `?`.
2. Trim and collapse whitespace.
3. Apply Unicode NFD.
4. Remove combining marks.
5. Uppercase using Locale.ROOT.
6. Replace characters outside [A-Z0-9] with spaces.
7. Trim and collapse spaces again.

Different accents or letter case may map to the same ID. The first inserted display string is retained.

6.6. Atomic writing

Write the complete dictionary to `names.pcn.tmp`, flush and close it, then atomically replace the old file when supported. Never patch the dictionary in place.

7. games.pcg - payload stream

`games.pcg` has no global header. It is a concatenation of payload records whose boundaries come only from `payloadOffset` and `payloadLength` in the index.

- Do not scan `games.pcg` sequentially to discover games.

- Old payloads can become orphaned after updates.
- Do not assume every byte is reachable.
- The record CRC covers the complete referenced payload.

7.1. Per-payload PCG1 header

Offset	Size	Type	Content
0	4	int32 BE	magic 0x50434731, ASCII PCG1
4	1	uint8	payload version = 1
5	1	uint8	codec: 1 MOVE16, 2 PGN_DEFLATE
6	2	uint16 BE	reserved = 0

```

string exactDate
string round
string fen
unsigned-varint extraTagCount
repeat extraTagCount:
    string tagName
    string tagValue
codec-specific body

```

7.2. String encoding and custom tags

```

unsigned-varint utf8ByteLength
byte[utf8ByteLength] UTF-8

```

Null is encoded as an empty string. Standard tags such as Event, Site, Date, Round, White, Black, Result, Elo, ECO, FEN, SetUp, and PlyCount must not be duplicated in extra tags.

8. Payload codecs

8.1. MOVE16_V1

MOVE16 stores 16-bit move codes and reconstructs SAN from board state. Use it only when the main line can round-trip safely.

- The start FEN is valid, or the standard initial position is used.
- Every main-line SAN token parses.
- There are no brace comments, semicolon comments, variations, or NAGs.
- No structure causes parser failure.
- Any failure must fall back to PGN_DEFLATE.

```

unsigned-varint moveCount
repeat moveCount:
    uint8 lowByte
    uint8 highByte

```

Bits	Meaning
0-5	from square 0..63
6-11	to square 0..63
12-14	promotion code
15	unused

Promotion	Piece
0	None
1	Knight
2	Bishop
3	Rook
4	Queen

```
code = (from & 63) | ((to & 63) << 6) | (promotionCode << 12)
```

Square mapping: The file format exposes only values 0..63. Non-PCOD implementations must verify the internal convention with e2-e4, g1-f3, castling, and promotion test vectors before releasing a writer.

Reconstructing SAN

8. Initialize the board from FEN or the standard start.
9. Unpack from/to/promotion.
10. Generate SAN from current board state.
11. Apply the move.
12. Add move numbers.
13. Append Result from the index.

8.2. PGN_DEFLATE_V1

This codec preserves UTF-8 movetext when the game has comments, variations, NAGs, graphical annotations, sound directives, complex or invalid SAN, or an invalid FEN.

```
unsigned-varint rawUtf8Length
unsigned-varint compressedLength
byte[compressedLength] zlib-compressed data
```

- The container is a zlib stream, not raw DEFLATE and not gzip.
- The reference compressor uses Java Deflater BEST_SPEED.
- Limit rawUtf8Length before inflating.
- Ensure compressedLength stays inside the payload boundary.
- The inflated byte count must exactly match rawUtf8Length.

9. Reconstructing complete PGN

A reader combines index fields, dictionary values, and decoded payload fields. PCOD writes tags in this order:

```
Event
Site
Date
Round
White
Black
Result
WhiteElo      when known
BlackElo      when known
ECO           when known
SetUp         when FEN exists
FEN           when present
custom tags   in payload order
```

9.1. Pseudocode

```

record = readIndex(gameId)
assert !record.deleted
payload = read(games.pcg, record.payloadOffset, record.payloadLength)
assert CRC32(payload) == record.payloadCrc
decoded = decodePayload(payload, resultText(record.resultCode))

tags.Event = dictionary.EVENT[record.eventId]
tags.Site = dictionary.SITE[record.siteId]
tags.Date = decoded.date != "" ? decoded.date : unpackDate(record)
tags.Round = decoded.round != "" ? decoded.round : "?"
tags.White = dictionary.PLAYER[record.whiteId]
tags.Black = dictionary.PLAYER[record.blackId]
tags.Result = resultText(record.resultCode)

append optional Elo/EC0/FEN and custom tags
write blank line + movetext

```

9.2. Escaping and annotations

```

\ -> \\
" -> \"

```

Type	Storage form
Comments	{...} or ;...
NAG	\$<number>
Variations	RAV in (...)
Diagram	comment beginning with # or [%diagram]
Arrows	[%cal <items>]
Colored squares	[%csl <items>]
Audio	[%pcod_sound filename.ext]

Annotation color codes are G green, R red, Y yellow, B blue, and O orange. An arrow item looks like `Ge2e4`; a square item looks like `Yd4`.

10. Audio assets and H2 sidecar

10.1. Audio assets

Audio bytes are not embedded in `games.pcg`. Movetext contains a reference while the actual file is stored under `sound/`.

```
1. e4 {[%pcod_sound <sha256>.mp3]} e5
```

- At most one audio reference per move node.
- Maximum source size is 50 MiB.
- Allowed extensions are mp3, flac, and ogg.
- The stored name is content SHA-256 plus lowercase extension.
- Accept safe basenames only; reject `..` and path traversal.
- A missing asset should not crash the core reader; report it as missing.

10.2. extras.mv.db

The H2 sidecar is optional. Core game reading must still work when the file or the H2 driver is unavailable.

```
jdbc:h2:file:<database>/extras;AUTO_SERVER=FALSE;DB_CLOSE_ON_EXIT=TRUE
```

Table	Purpose
SOURCE_IMPORT	Import source history, imported/skipped counts, timestamp

Table	Purpose
USER_NOTE	Per-Game-ID note, rating, and modification time

- Never make H2 the only source of game data.
- Do not require H2 for core PGN export.
- Version any new sidecar schema separately.

11. Safe read workflow

14. Confirm the root is a `.pcod`` directory.
15. Read Java Properties and validate format, version, and record size.
16. If a recovery marker exists, run compatible recovery or refuse read-write access.
17. Read the little-endian ``games.pci`` header and obtain committed gameCount.
18. Check the index is not truncated; handle trailing bytes according to open mode.
19. Read ``names.pcn`` using big-endian fixed fields, varints, and Java UTF-16 prefix semantics.
20. Read the exact 56-byte record for each valid Game ID.
21. Skip deleted records unless explicitly requested.
22. Validate offset/length, overflow, and containment within ``games.pcg``.
23. Read the payload and verify CRC32 before decode.
24. Validate PCG1, version, codec, and every field boundary.
25. Decode with resource limits.
26. Resolve stable IDs and reconstruct PGN.

11.1. Recommended read-only behavior

For inspection and export, open files read-only and do not silently truncate trailing data. Report a warning and let an official or compatible recovery tool make changes.

12. Import transactions and crash recovery

12.1. Import marker

```
recovery/import.properties
gameCount=<committed count before import>
payloadLength=<games.pcg length before import>
players=<player count before import>
events=<event count before import>
sites=<site count before import>
```

Early Hybrid 0.2 markers may omit dictionary counts. Index and payload can still roll back; unused names may remain harmlessly.

12.2. Safe import order

27. Atomically write and force the marker.
28. For each game, checkpoint payload length and dictionary counts.
29. Encode the payload and allocate name IDs.
30. Append the payload.
31. Append the index record in uncommitted space.
32. On per-game failure, truncate the payload and roll back that game's names.
33. Force games.pcg.
34. Force index record data.
35. Atomically save names.pcn.
36. Write committed gameCount at offset 16 and force it.
37. Delete the marker.

12.3. Recovery on open

```
truncate games.pci to gameCount
truncate games.pcg to payloadLength
roll back names to counts when present
atomically save names.pcn
delete the marker
```

Why the count is last: Before count advances, new physical records are uncommitted. After count advances, every referenced payload, index record, and dictionary entry must already be durable.

13. Update, soft delete, and compaction

13.1. Updating one game

`recovery/update.properties` stores the Game ID, payload length before the update, and all fields of the old index record.

38. Read the old record.
39. Write the update marker.
40. Encode and append the new payload.
41. Force the payload.
42. Write the new index record at the same Game ID.
43. Force the index.
44. Delete the marker.

On failure or crash, restore the old record, truncate the payload to the checkpoint, and remove the marker.

13.2. Current update limitation

The current replacement flow is designed mainly for movetext and annotations. It keeps IDs, packedDate, packedElo, result/ECO, and the deleted flag; it changes payload pointers, content flags, ply count, semantic hash, and CRC. Do not use it to replace the complete PGN header.

13.3. Compaction

45. Create a new bundle.
46. Scan active records.
47. Load complete PGN.
48. Import through the standard writer.
49. Copy still-referenced audio files.
50. Validate the new database.
51. Close both bundles.
52. Atomically rename or swap directories when supported.
53. Retain a backup of the old database.

Game IDs after compaction: Re-import can assign new Game IDs. Preserving IDs requires a specialized tool that retains deleted slots or publishes an ID mapping.

14. Validation and data safety

14.1. Minimum validator checklist

Area	Required checks
Manifest	Exists; correct format/version; record size 56
Index header	Magic/version/header/record size; nonnegative count; no

Area	Required checks
	truncation
Index record	Valid offset/length; name IDs exist; CRC matches; nonnegative plyCount
Dictionary	Magic/version; counts; varints; suffix; UTF-16 prefix; unique IDs
Payload	PCG1/version/codec; string/tag boundaries; MOVE16 count; zlib lengths
Sound	Safe basename; valid extension; asset exists when playback is required

14.2. Recommended resource limits

- Cap payloadLength by policy.
- Cap rawUtf8Length before inflation.
- Cap custom-tag count and every string length.
- Bound moveCount by remaining payload bytes.
- Use overflow-safe arithmetic for offset + length.
- Use strict UTF-8 decoding or document a clear replacement policy.

14.3. Backup scope

A complete backup copies the entire `.pcod`` directory, including manifest, index, dictionary, payload, sidecar, sound assets, and any recovery marker present. Copying only the three core binary files loses notes or audio.

15. Format conversion

15.1. PCOD to another format

54. Read the manifest.
55. Recover or refuse an unresolved marker.
56. Read the dictionary.
57. Iterate Game IDs from 1 to committed count.
58. Skip deleted records unless requested.
59. Verify CRC.
60. Decode the payload.
61. Reconstruct canonical PGN.
62. Convert the domain object to the target format.

15.2. Another format to PCOD

The safest first implementation is:

```
Other source -> normalized PGN -> import through PCOD core
```

A direct binary writer must implement manifest, dictionaries, codecs, CRC, semantic hash, index packing, atomic files, recovery, and crash simulation. Never copy an index record between databases because its name IDs belong to a different dictionary.

16. Python index-reader skeleton

This read-only skeleton validates and reads the index and payload bytes. It is not a complete writer and does not yet decode dictionaries or payload bodies.

```

from dataclasses import dataclass
from pathlib import Path
import struct
import zlib

PCI_MAGIC = 0x50434931
INDEX_HEADER_SIZE = 64
INDEX_RECORD_SIZE = 56

@dataclass(frozen=True)
class IndexRecord:
    payload_offset: int
    payload_length: int
    white_id: int
    black_id: int
    event_id: int
    site_id: int
    packed_date: int
    packed_elo: int
    packed_meta: int
    ply_count: int
    semantic_hash: int
    payload_crc: int

    @property
    def deleted(self) -> bool:
        return bool(self.packed_meta & (1 << 12))

RECORD_STRUCT = struct.Struct("<qiiiiiiiQI")

def read_index_header(path: Path) -> int:
    with path.open("rb") as f:
        raw = f.read(INDEX_HEADER_SIZE)
    if len(raw) != INDEX_HEADER_SIZE:
        raise ValueError("truncated games.pci header")
    magic, version, record_size, header_size, _ = struct.unpack_from(
        "<IHHII", raw, 0
    )
    game_count = struct.unpack_from("<Q", raw, 16)[0]
    if magic != PCI_MAGIC:
        raise ValueError("invalid PCI magic")
    if version != 1 or record_size != 56 or header_size != 64:
        raise ValueError("incompatible index")
    return game_count

def read_index_record(path: Path, game_id: int) -> IndexRecord:
    if game_id < 1:
        raise ValueError("Game ID starts at 1")
    offset = INDEX_HEADER_SIZE + (game_id - 1) * INDEX_RECORD_SIZE
    with path.open("rb") as f:
        f.seek(offset)
        raw = f.read(INDEX_RECORD_SIZE)
    if len(raw) != INDEX_RECORD_SIZE:
        raise ValueError("truncated index record")
    return IndexRecord(*RECORD_STRUCT.unpack(raw))

def read_payload(path: Path, record: IndexRecord) -> bytes:
    if record.payload_offset < 0 or record.payload_length < 0:
        raise ValueError("invalid offset/length")
    with path.open("rb") as f:
        f.seek(record.payload_offset)
        data = f.read(record.payload_length)
    if len(data) != record.payload_length:
        raise ValueError("truncated payload")
    if zlib.crc32(data) & 0xFFFFFFFF != record.payload_crc:
        raise ValueError("payload CRC mismatch")
    return data

```

Unsigned semantic hash: Python ``q`` is signed int64. To display semanticHash as unsigned, use ``value & 0xFFFFFFFFFFFFFFFF``.

17. Cheat sheets and troubleshooting

17.1. Mixed-endian cheat sheet

Component	Endianness / encoding
games.pci header	Little-endian
games.pci record	Little-endian
names.pcn fixed fields	Big-endian
names.pcn varint	7-bit, low groups first
names.pcn suffix	UTF-8
PCG1 fixed header	Big-endian
PCG1 string lengths	Varint
MOVE16 code	Low byte then high byte
DEFLATE lengths	Varint
Compressed body	zlib stream

17.2. Common failures

Symptom	Likely cause	Remedy
Nonsensical IDs/offsets	Wrong endianness	Follow the per-file endian table
Broken name prefix	Prefix counted as bytes/code points	Reproduce Java UTF-16 code units
CRC mismatch	Wrong boundary or corruption	Do not decode; report integrity failure
Gzip decoder fails	Payload uses zlib, not gzip	Use a zlib wrapper
Comments/variations lost	MOVE16 forced	Fall back to PGN_DEFLATE
Game IDs shifted	Zero-based counting or ID reuse	Game ID = ordinal + 1; preserve deleted slots
Wrong names after record copy	IDs belong to another dictionary	Export PGN and re-import
Payload file keeps growing	Orphans after update	Compact into a new bundle

17.3. Invariants to preserve

- 64-byte index header and 56-byte records.
- Stable Game IDs and header count as commit point.
- Deletion only sets a flag.
- Append-only payloads.
- Explicit stable dictionary IDs.
- Section order PLAYER, EVENT, SITE.
- CRC over the complete payload.
- MOVE16 only when round-trip safe.
- Invalid FEN preserved through PGN_DEFLATE.
- Audio reference in PGN and content in `sound`.
- H2 is not core storage.
- Every writer transaction uses a recovery marker.

18. Future compatibility

- Reject unsupported format versions instead of guessing.
- Ignore reserved fields/bits only when boundaries remain unambiguous; warn on unusual values.
- Do not require reserved values to be zero for reading.
- Do not delete unknown files while copying or backing up a bundle.
- A 0.2 writer must not add fields to the 56-byte record.
- Put new metadata in custom payload tags, a versioned sidecar, or a new format version.
- When the format changes, publish a new document such as ``PCOD_DATABASE_FORMAT_0.3_FOR_CHATGPT.md`` and retain the 0.2 guide for old databases.

18.1. Release checklist

Group	Checklist
Reader	Version; endian; CRC; limits; soft delete; UTF-16 prefix; audio sidecar
Writer	Recovery; force order; atomic dictionary; per-game rollback; round-trip tests
Tests	Unicode; invalid FEN; variations; NAG; audio; large database; crash at every commit stage
Release	No record-size change; no ID reuse; preserve unknown files; documented backup

19. MIT License

The author and copyright holder permits use of this format documentation under the MIT License. The grant applies to this guide and associated specification files, not automatically to separately distributed software or brand assets.

19.1. Practical meaning

- You may use, copy, modify, merge, publish, distribute, sublicense, and sell copies of the documentation.
- You must retain the copyright and permission notice in copies or substantial portions.
- The documentation is provided “as is,” without warranty.
- The authors and copyright holders are not liable for claims or damages arising from its use.

19.2. Full license text

MIT License

Copyright (c) 2026 SpiderPawn Việt Nam

Permission is hereby granted, free of charge, to any person obtaining a copy of this documentation and associated specification files (the "Documentation"), to deal in the Documentation without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Documentation, and to permit persons to whom the Documentation is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Documentation.

THE DOCUMENTATION IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE DOCUMENTATION OR THE USE OR OTHER DEALINGS IN THE DOCUMENTATION.

End of document.